Addressing Modes

The various formats of representing operand in an instruction or location of an operand is called as "Addressing Mode". The different types of Addressing Modes are

a) Register Addressing

b) Direct Addressing

c) Immediate Addressing

d) Indirect Addressing

e) Index Addressing

f) Relative Addressing

g) Auto Increment Addressing

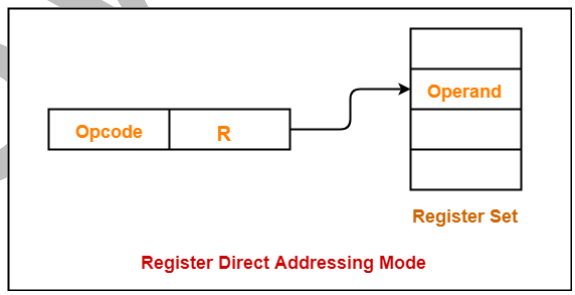
h) Auto Decrement Addressing

a) Register Addressing:

In this mode operands are stored in the registers of CPU. The name of the register is directly specified in the instruction.

Ex: MOVE R1, R2

Where R1 and R2 are the Source and Destination registers respectively. This instruction transfers 32 bits of data from R1 register into R2 register. This instruction does not refer memory for operands. The operands are directly available in the registers.

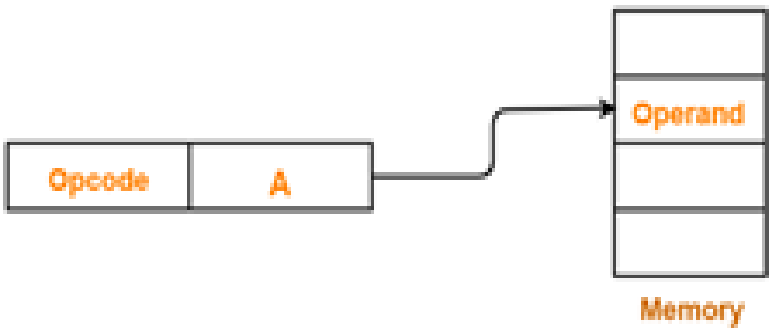


b) Direct Addressing

It is also called as Absolute Addressing Mode. In this addressing mode operands are stored in the memory locations. The name of the memory location is directly specified in the instruction.

Ex: MOVE LOCA, R1: Where LOCA is the memory location and R1 is the Register.

This instruction transfers 32 bits of data from memory location X into the General Purpose Register R1.

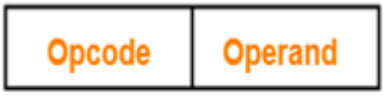


Direct Addressing Mode

c) **Immediate Addressing**

In this Addressing Mode operands are directly specified in the instruction. The source field is used to represent the operands. The operands are represented by # (hash) sign.

Ex: MOVE #23, R0



Immediate Addressing Mode

d) **Indirect Addressing**

In this Addressing Mode effective address of an operand is stored in the memory location or General Purpose Register. The memory locations or GPR^S are used as the memory pointers.

Memory pointer: It stores the address of the memory location.

There are two types Indirect Addressing

i. Indirect through GPR^S

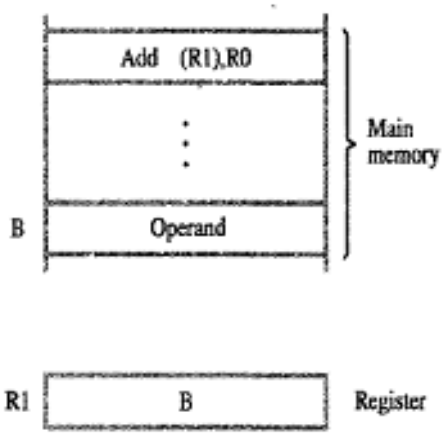
ii. Indirect through memory location

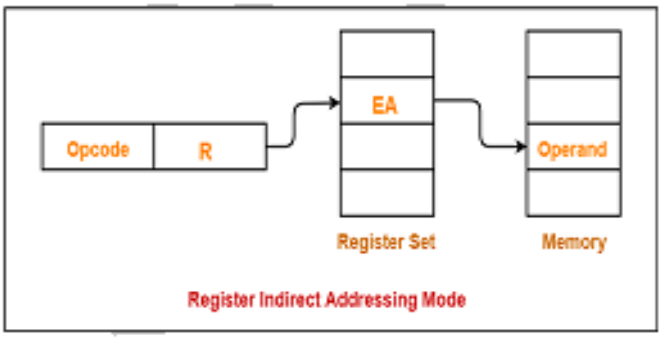
i)Indirect Addressing Mode through GPR^S.

In this Addressing Mode the effective address of an operand is stored in the one of the General Purpose Register of the CPU.

Ex: ADD (R1), R0 ; Where R1 and R0 are GPR^S.

This instruction adds the data from the memory location whose address is stored in R1 with the Contents of R0 Register and the result is stored in R0 register as shown in the fig.





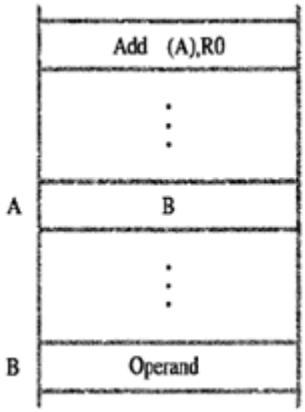
ii)Indirect Addressing Mode through Memory Location.

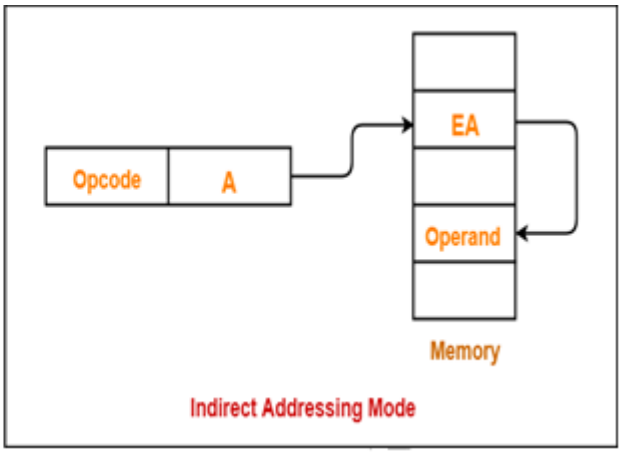
In this Addressing Mode, effective address of an operand is stored in the memory location.

Ex: ADD (X), R0

This instruction adds the data from the memory location whose address is stored in 'X' memory location with the contents of R0 and result is stored in R0 register.

The diagrammatic representation of this addressing mode is as shown in the fig





e) **Index Addressing Mode**

In this addressing mode, the effective address of an operand is computed by adding constant value with the contents of Index Register and any one of the General Purpose Register namely R0 to Rn-1 can be

used as the Index Register. The constant value is directly specified in the instruction.

The symbolic representations of this mode are as follows

1. X (R_i) where X is the Constant value and R_j is the GPR.

It can be represented as

EA of an operand = X + (R_i)

2. X (R_i , R_j) Where X is the constant value and R_I and R_J are the General Purpose Registers used to store the addresses of the operands. It can be represented as

The EA of an operand is given by EA = (R_i) + (R_j) + X

f) **Relative Addressing Mode**

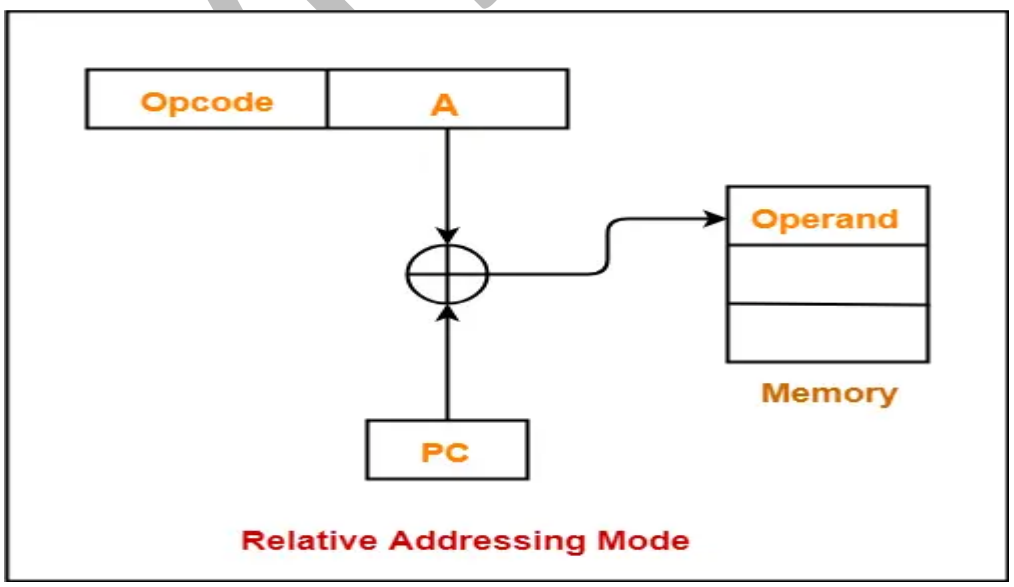
In this Addressing Mode EA of an operand is computed by the Index Addressing Mode. This Addressing Mode uses PC (Program Counter) to store the EA of the next instruction instead of GPR.

The symbolic representation of this mode is X (PC).Where X is the offset value and PC is the Program Counter to store the address of the next instruction to be executed.

It can be represented as

EA of an operand = X + (PC).

This Addressing Mode is useful to calculate the EA of the target memory location.



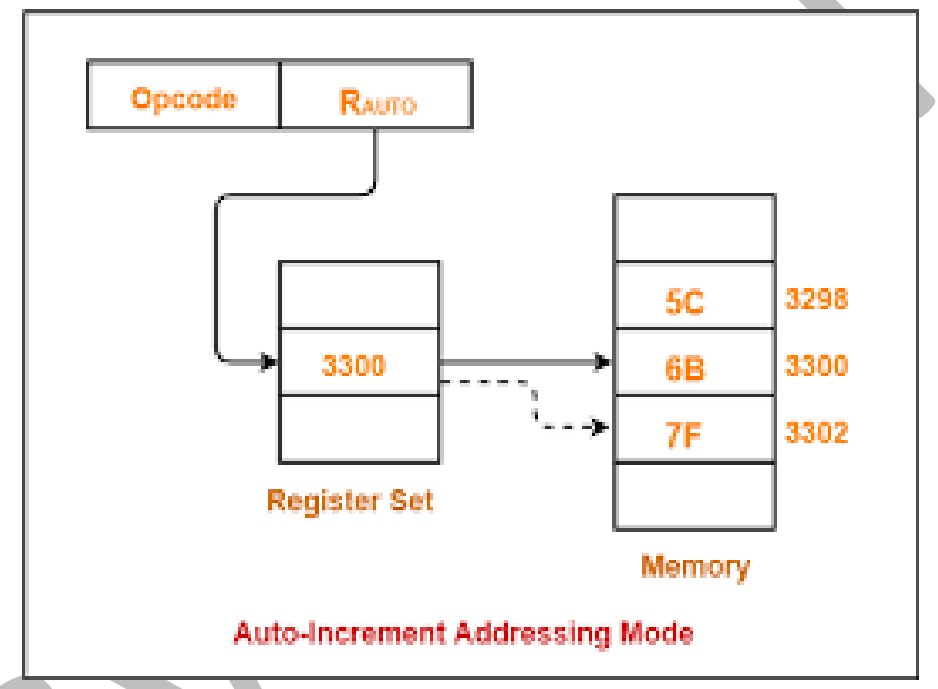
g) **Auto Increment Addressing Mode**

In this Addressing Mode, EA of an operand is stored in the one of the GPR^S of the CPU. This Addressing Mode increment the contents of memory register by 4 memory locations after operand access. The symbolic representation is

$(R_I)+$ Where R_i is the one of the GPR.

Ex: MOVE $(R_1)+, R_2$

This instruction transfer's data from the memory location whose address is stored in R_1 into R_3 register and then it increments the contents of R_1 by 4 memory locations



h) **Auto Decrement Addressing Mode**

In this Addressing mode, EA of an operand is stored in the one of the GPR^S of the CPU. This Addressing Mode decrements the contents of memory register by 4 memory locations and then transfers the data to destination.

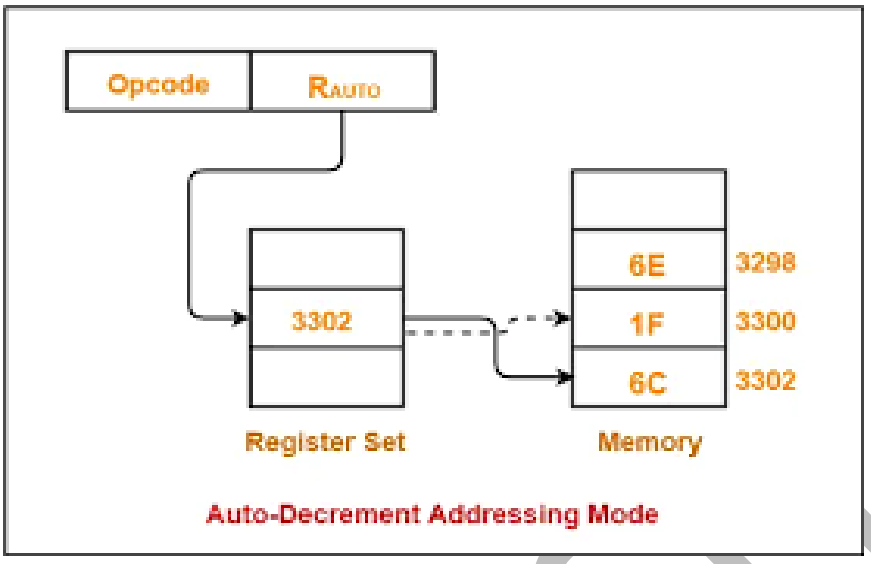
The symbolic representation is

$-(R_I)$ Where R_i is the one of the GPR.

Ex: MOVE $-(R_1), R_2$

This instruction first decrements the contents of R_1 by 4 memory locations and then transfer's data

of that location to destination register.



Assembly Language

- The Assembly language uses Symbolic names to represent opcodes ,memory locations and registers.
- The Assembler converts Assembly language programs into machine level language programs.
- The Assembly language is called as "Source program" and m/c language program is called As "Object program".

The Assembler converts source program into object program.

A set of symbolic names and set of rules for their use forms a programming language and is called as "Assembly Language"

Ex: MOV, ADD, LOAD → Opcodes.

X, Y, AMOUNT → Memory locations.

Where R0 to R7 are the registers.

The examples for Assembly Language instructions are as follows

MOV R0,R1 ; Register Addressing.

MOV #23, R0 ; Immediate Addressing.

Assembler Directives

There are two types of instructions namely i) Processor Instructions and ii) Assembler Instructions.

- The Processor instructions are converted into m/c instructions by means of Assembler. Hence Assembler generates m/c code for processor instructions.
- The Assembler instructions are not converted into m/c instructions and hence Assembler does not generate the m/c code for Assembler instructions

"A set of commands given to Assembler while converting source program into object program is called as Assembler Directive".

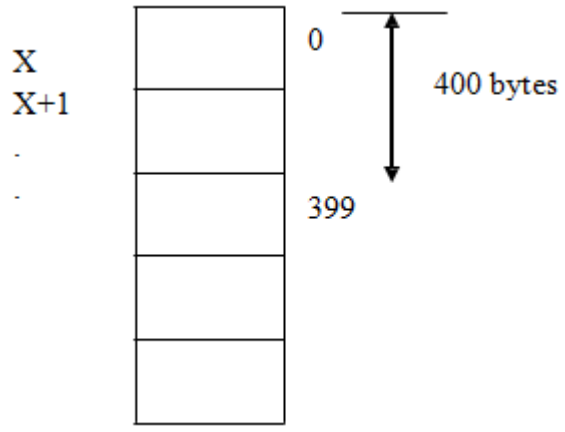
Types of Assembler Directives:

1) **RESERVE**

This directive is used to allocate a block of memory. This block of memory is used only for data.

X RESERVE 400

This directive reserves 400 bytes of memory whose symbolic name is X as shown in fig



2) **EQU**

This directive is used to assign numerical values to symbolic names during the execution of the assembly language program.

The following code describes the use of EQU directive.

X EQU 32

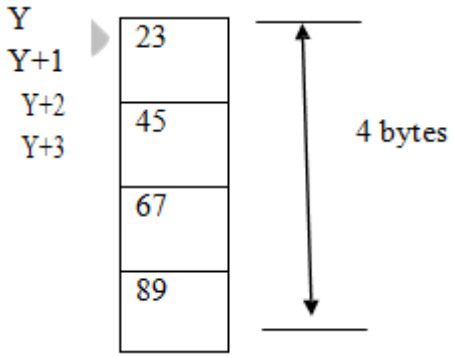
```
MOVE    #23,    R0
MOVE    X,      R1
ADD     R0,     R1
```

3) **DATA WORD**

This directive is used to allocate 4 bytes of memory.

Y DATAWORD 23456789

The memory representation of this directive is as shown in the fig.



4) ORIGIN

This directive converts source program into object program. The object program is loaded into memory for execution by means of loader. The Origin directive is used to assign the successive addresses for sequence of instructions or operands.

ORG 100

5) **END**

This Directive is used to terminate the Assembly level language program. The Assembler ignores the execution of instructions after the execution of this directive.

END

Basic Input and Output Operations:

The simple arrangement of connecting i/p and o/p devices into the processor is as shown in the fig

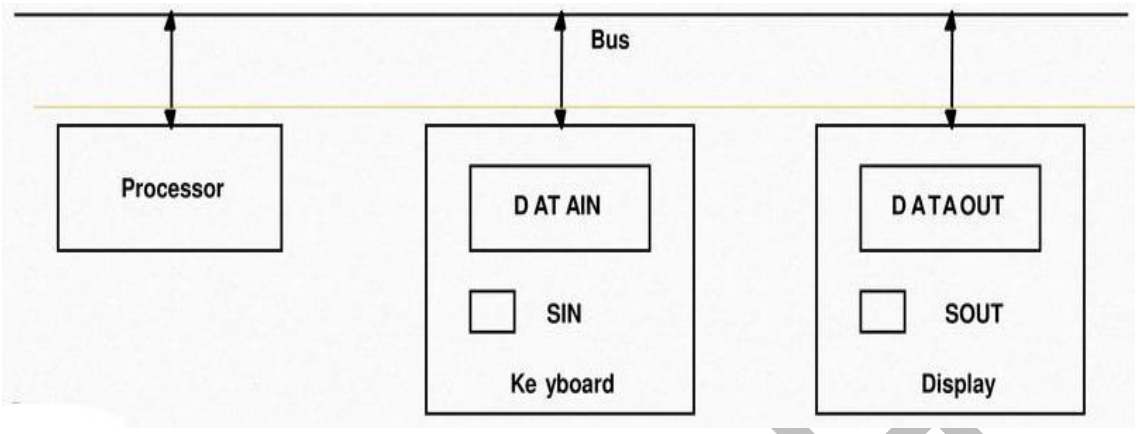
The Processor performs two operations with respect to i/o device namely

i) Input operation

ii) Output operation

Input operation: It is the process of reading the data or instructions from the input device. The I/O subsystem consists of block of instructions to perform i/p operation.

Output operation: It is the process of writing the data or instructions into the output device. The I/O subsystem consists of block of instructions to perform o/p operation



Consider a problem of transferring 1 byte of data from i/p device to o/p device. The i/p device transfers few characters/sec. The data transfer rate of i/p device is expressed in terms of few characters/sec. Similarly o/p device transfers thousands of characters to o/p device for display. The processor is capable of executing millions of instructions per second. From the above analysis it is clear that the processing and transfer speed varies in different devices. So the devices must be synchronized.

To provide the synchronization between Processor, i/p device and o/p device it is necessary to follow the several steps are as follows

Steps to provide synchronization between Processor and i/p device

1.When a character is pressed on the keyboard, the ASCII value of a character is stored in DATAIN register and hence SIN flag is set to 1.

2.When SIN = 1, the processor reads the ASCII value of a character from DATAIN register into the Processor register.

3.After reading, the SIN flag is reset to 0.

```
READWAIT    if SIN = 0
            Branch  to READWAIT              //No data to read
            Input data from DATAIN to R1  //Data is read
```

Steps to provide synchronization between Processor and O/p device

1. When the o/p device is ready to display the character, the Processor transfers the character code from the processor register into the DATAOUT register.
2. The SOUT flag is set to 1 when DATAOUT register holds the character code.

3. The SOUT flag is cleared when the character code is transferred to o/p device.

```
WRITEWAIT    if SOUT = 0
             Branch  to WRITEWAIT          //No data to read
             Input data from R1 to DATAIN  //Data is
```

The i/p operation can be implemented as follows

Let R0 be the Processor register and DATAIN be the internal register of the i/p device

MOVE DATAIN, RO

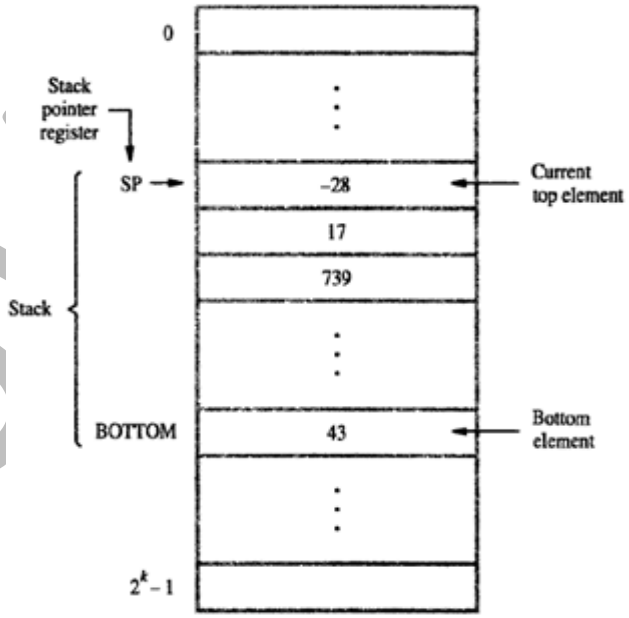
The o/p operation can be implemented as follows

Let R0 be the Processor register and DATAOUT be the internal register of the O/p device.

MOVE RO, DATAOUT

STACKS AND QUEUES

STACK - A stack is a Data Structure, in which the accessing is restricted at only one end of the stack. It is similar to a bottle, in which elements can be added and removed from the same end. The end of the stack, from which elements can be added or removed, is called the top of the stack and the other end is called the bottom of the stack.



It works on the principle of LIFO (Last In First Out), the last item placed on the stack is the first to be removed. The term 'push' and 'pop' are used to describe the placing a new item on stack and removing the top item from the stack.

Assume that the first element is placed in the location BOTTOM, and when they are placed in successive lower addresses.

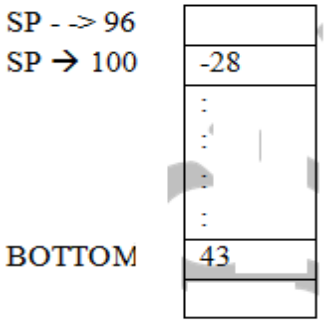
A processor register is used to keep track of the address of the element that is at the top of stack at any time. This register is called Stack Pointer **(SP).**

In the above figure, the SP pointer is currently pointing to the topmost value -28. To add a new element, the SP will decrement its value by 1 address, so as to point at next location and add the new value.

PUSH Operation – Subtract #4, SP

MOV NEWITEM, (SP)

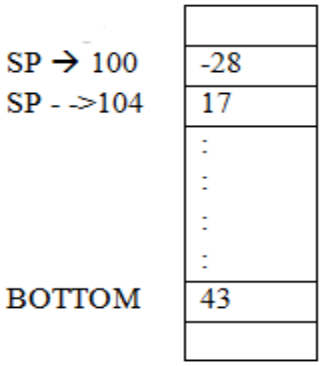
The subtract instruction subtracts the SP value by 4, now SP points to the next lower address. The MOV instruction moves the new element to the address location stored in SP.



POP Operation -MOV (SP), ITEM

ADD #4, SP

The MOV instruction moves the element at the location pointed by SP to ITEM and the SP pointer is moved to the next higher address, so that it points to the new top element.



QUEUE – A queue is a Data Structure that works on the principal of FIFO (First In First Out) ie., data that are stored first are retrieved first on FIFO basis.

The elements are added at one end (IN) and retrieved from other end (OUT). In stack, one end is fixed where as in queue both ends are pointed by pointers and both end changes its location. One end is used to add items and other end is used to delete items.

Subroutine

Sub functions in a program necessary to perform a particular subtask are called a subroutine.

Eg: Subroutine to sort a list of numbers, Subroutine to add the given numbers etc.

In a program, the subroutines an be called from different locations and different functions. When a program branches to a subroutine, then it is calling the subroutine. The instruction that perform this branch operation is called **call instruction.**

Whenever the subroutine is called, the execution starts from the starts from the starting address of subroutine. After its execution, the execution of calling function is resumed from the location where it called the subroutine. Hence the content of PC is stored before moving to the subroutine.

The way in which a computer calls and returns from a subroutine are called subroutine linkage method. The return address is stored in link register. After the execution of subroutine, the return instruction returns to the calling program by using the link register.

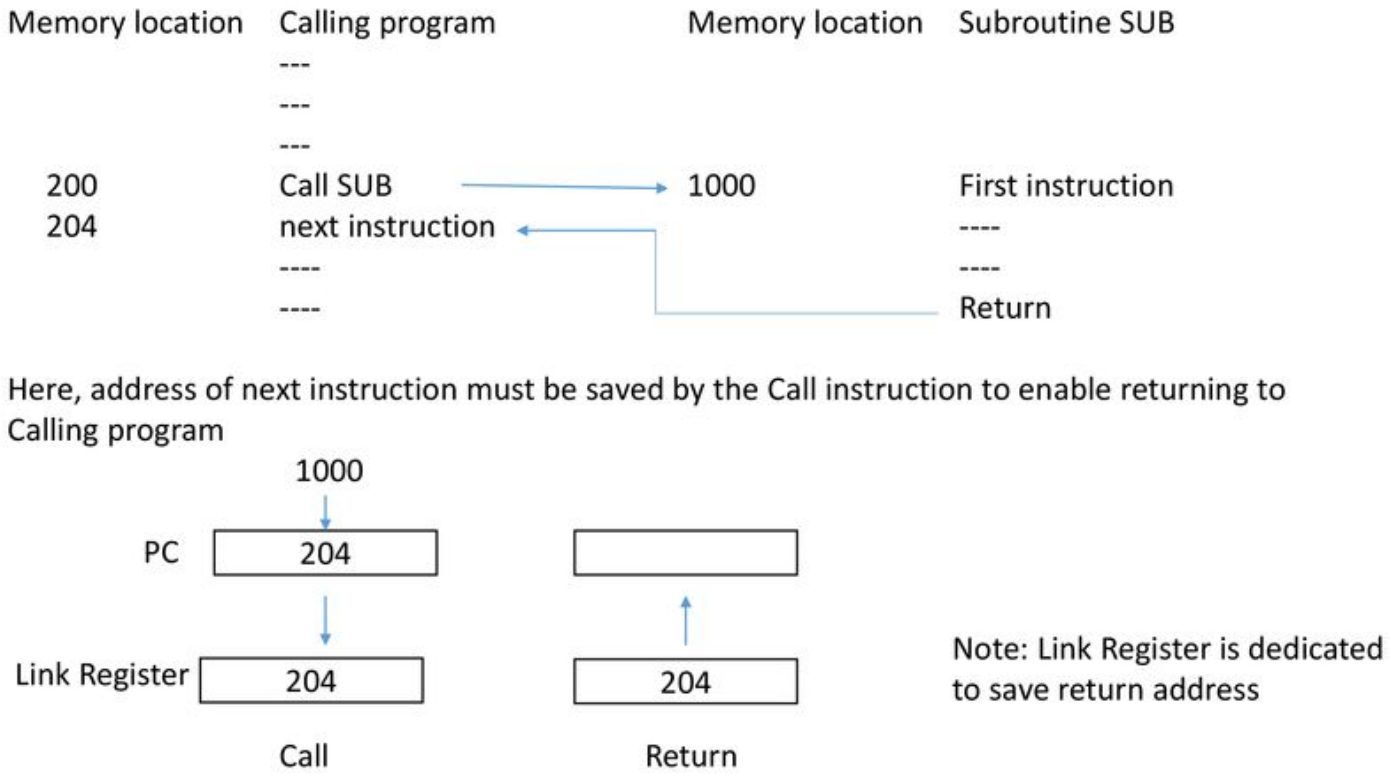


Figure Subroutine linkage using a link register

The Call instruction is a special branch instruction –

- It stores the content of PC in link register
- Stores the specified subroutine address in PC and branch to that address.

The Return instruction of subroutine is a special branch instruction –

- It branches to the address contained in link register.

Parameter Passing: Example for subroutine (Adding a list of numbers)

Calling program	MOVE N, R1
	MOVE #NUM1, R2
	CALL LISTADD
	MOVE R0, SUM
	…
	…
	…

Subroutine	
LISTADD	CLEAR 0 ADD (R2)+, R0 DECREMENT R1 BRANCH>0 LOOP RETURN

Multiplication and Division:

Format: opcode source operand, destination operand

Ex: Multiply Ri, Rj ; Ri = [Ri] * [Rj]

Multiply R3, R0

Performs the operation as R0 = [R3] * [R0]

Ex: Divide Ri, Rj ; Rj = [Rj] / [Ri]

Divide R3, R0

Performs the operation as R0 = [R0] / [R3]

Additional Instructions

There are 3 types

a) Logical instructions – NOT, AND , OR

b) Shift instructions – Logical Shift Left, Logical Shift Right, Arithmetic Shift right

c) Rotate instructions – Rotate Left with carry, Rotate Left without carry,

Rotate Right with carry, Rotate Right without carry.

Logical instructions

The processors are designed to perform logical operations such as AND,OR and NOT operations.

i) NOT instruction

Format: opcode destination

The opcode specifies operation to be performed and destination specifies the operand. The operand can be register operand or memory operand.

Ex : NOT R0

It performs the function of complementation. It is the process of converting binary bit 0 into binary bit 1 and vice versa.

The illustration of this instruction is as follows.

Before instruction execution	**After instruction execution**
R0 = 10101100	R0 = 01010011.

ii) AND instruction

It performs the function of logical AND operation.

Format: opcode source, destination

Ex: AND R3, R0

This instruction logically ANDs the contents of R3 with the contents of RO and result is stored in R0 register.

iii) OR instruction

It performs the function of logical OR operation.

Format: opcode source, destination

Ex: OR R3, R 0

This instruction logically OR[S] the contents of R3 with the contents of RO and result is stored in R0 register.

Shift instructions

The shift instructions are designed to shift the contents of processor register or memory location to left or right according to the number of bits specified in the count.

There are 2 types of shift instructions.

1. Logical Shift Left.
2. Logical Shift Right.
3. Arithmetic Shift Right

Logical Shift Left:

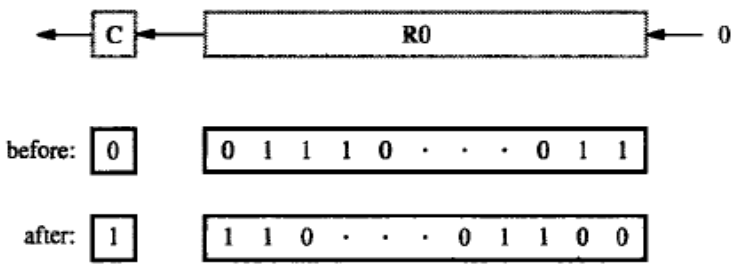
Format: opcode count, destination

The opcode indicates operation to be performed. The count can be either immediate operand or the contents of processor register. The destination can be either register operand or memory operand.

Ex: LshiftL #2, R0

This instruction shifts the contents of register R0 to left through carry by 2 bits. The count value directly specified in the instruction as an immediate operand.

The contents of R0 before and after the execution of this instruction are as shown in the fig. The **shifted positions are filled with zeros from right side** as shown in the fig.



Logical Shift Right:

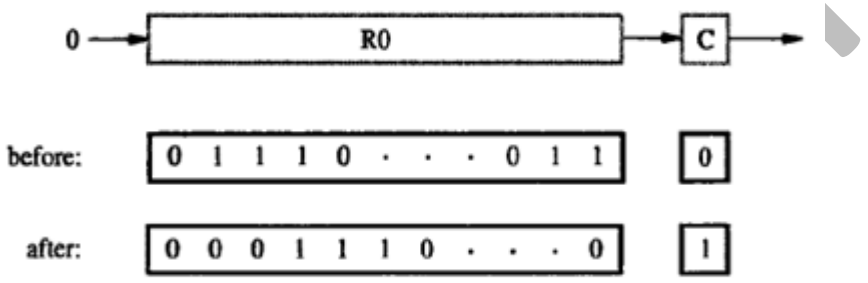
Format: opcode count, destination

The opcode indicates operation to be performed. The count can be either immediate operand or the contents of processor register. The destination can be either register operand or memory operand.

Ex: LshiftR #2, R0

This instruction shifts the contents of register R0 to right through carry by 2 bits. The count value directly specified in the instruction as an immediate operand.

The contents of R0 before and after the execution of this instruction are as shown in the fig. The **shifted positions are filled with zeros from left side** as shown in the fig.



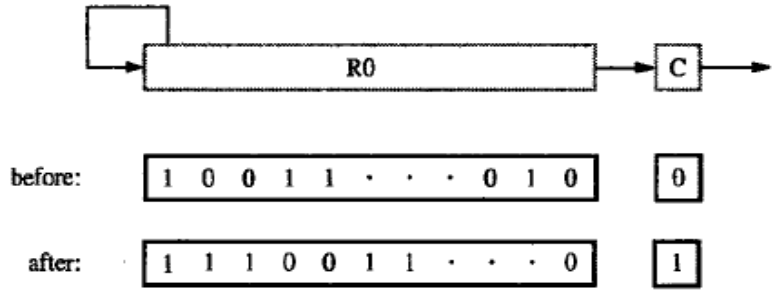
Arithmetic Shift Right:

Format: opcode count, destination

The opcode indicates operation to be performed. The count can be either immediate operand or the contents of processor register. The destination can be either register operand or memory operand

Ex: AShiftR #2, R0

This instruction is designed **to preserve the sign bit**. This instruction shifts the contents of register or memory location to **right through carry**, by number of bits specified in the count. After each shift it **copies leftmost bit to Most Significant Bit**. The contents of R0 before and after the execution of this instruction are as shown in the fig.



Rotate instructions

The Rotate instructions are designed to rotate the contents of register or memory location to left or right according to the number of bits specified in the count.

There are 4 types of rotate instructions.

1. Rotate left without carry
2. Rotate left with carry
3. Rotate right without carry
4. Rotate right with carry

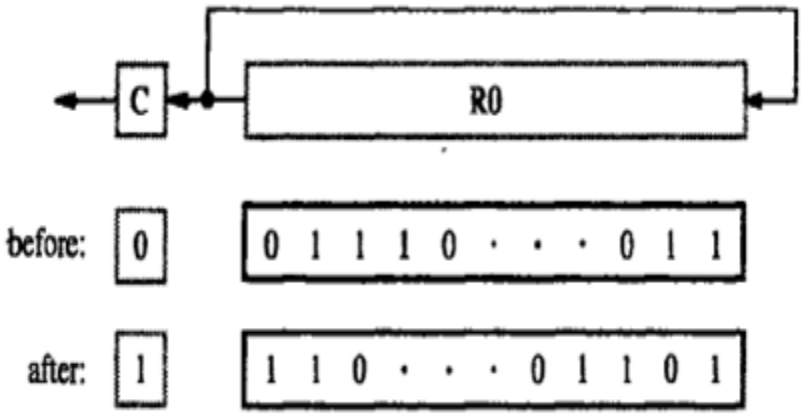
Rotate left without carry:

Format: opcode count, destination

The opcode indicates operation to be performed. The count can be either immediate operand or the contents of processor register. The destination can be either register operand or memory operand.

Ex: RotateL #2, R0

This instruction rotates the contents of register R0 **to left without carry** by 2 bits as shown in the fig. The Most Significant Bits are transferred to Least Significant Bits are as shown in the fig. The contents of register R0 before and after the execution of this instruction is as shown in the fig.



Rotate left with carry:

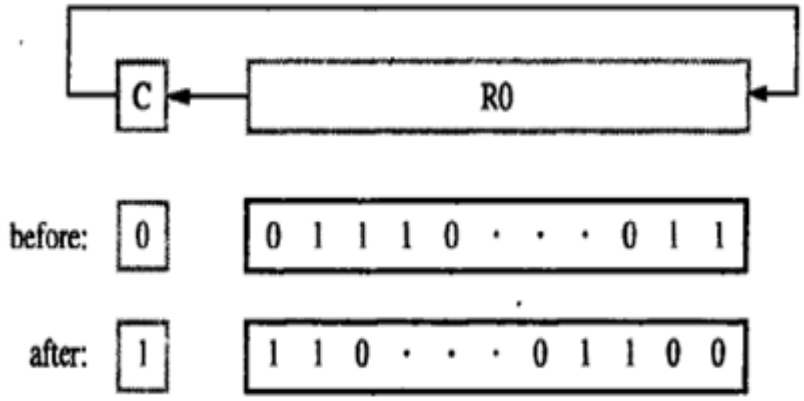
Format: opcode count, destination

The opcode indicates operation to be performed. The count can be either immediate operand or the

contents of processor register. The destination can be either register operand or memory operand.

Ex: RotateLC #2, R0

This instruction rotates the contents of register R0 **to left with carry** by 2 bits as shown in the fig. **The Most Significant Bits are transferred to carry and then transferred to Least Significant Bits** are as shown in the fig. The contents of register R0 before and after the execution of this instruction is as shown in the fig.



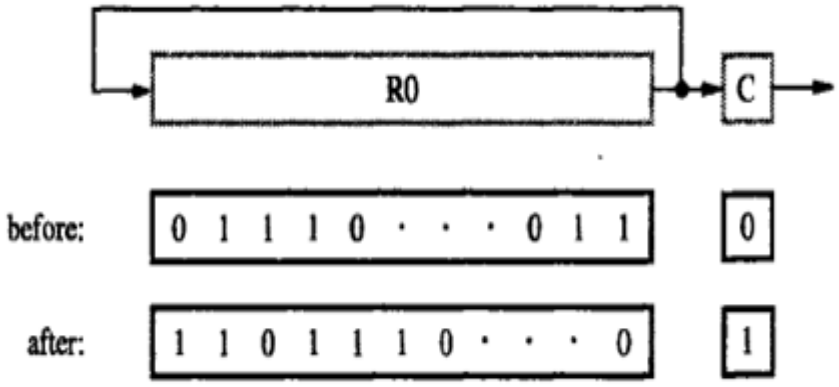
Rotate right without carry

Format: opcode count, destination

The opcode indicates operation to be performed. The count can be either immediate operand or the contents of processor register. The destination can be either register operand or memory operand.

Ex: RotateR #2, R0

This instruction rotates the contents of register R0 to right without carry by 2 bits as shown in the fig. **The Least Significant Bits are transferred to Most Significant Bits** are as shown in the fig. The contents of register R0 before and after the execution of this instruction are as shown in the fig.



Rotate right with carry

Format: opcode count, destination

The opcode indicates operation to be performed. The count can be either immediate operand or the contents of processor register. The destination can be either register operand or memory operand.

Ex: RotateRC #2, R0

This instruction rotates the contents of register R0 to **right with carry** by 2 bits as shown in the fig. **The Least Significant Bits are transferred to carry and then transferred to Most Significant Bits** are as shown in the fig. The contents of register R0 before and after the execution of this instruction are as shown in the fig.